What Catches a Lie That Works? Trust Scores, Anomaly Gates and Load-Misreporting Servers in Zero-Trust Edge Load Balancing

K R Jeevan Raj, Aravind Kumar M, Arjun Krishnakumar, Ashwin Kumar RM, Bhuvanesh S
Department of Computer Science and Engineering
Amrita Vishwa Vidyapeetham, Coimbatore, India
Faculty guide: Dr. Radhika N, Professor, Department of CSE

***Abstract*—**Zero Trust demands continuous verification but does not say how a control plane should verify the servers it routes work to. We built a Zero-Trust SDN load balancer in which a trust score ranks edge servers, an independent anomaly gate excludes them, and every verdict is installed as OpenFlow 1.3 rules, and evaluated it live on an emulated 8-server, 40-device edge. In a five-run-per-arm ablation the score never isolated two attackers that complete every task while the gate isolated both every time; a threshold sweep shows the score can, but only within a band a few hundredths wide and 3.4–20× more slowly. Those attackers were slow, so the gate caught slowness, not a lie. In 80 further runs, each against a prediction registered first, a liar serving at honest speed with internally consistent claims gains nothing, while one that lies only in the instantaneous load figure the router ranks on, with a truthful busy-time counter, wins 4.0 points of excess traffic and is never caught; ranking on the verified figure closes the hole at no measured honest cost. Emulated mixed hardware is not mistaken for lying; switch counters joined with client reports name the lying party in 25 of 25 runs; attestation stops a tampered server before it serves; and ledger replicas that replay the trust arithmetic reject every forged score, yet a controller forging evidence passes them and blinds its own detectors. Only the switches expose it; acting on their verdicts restores honest task success from 82% to 99%.

***Index Terms*—**Zero Trust, software-defined networking, edge computing, trust-aware load balancing, ablation study, OpenFlow, anomaly detection

I. Introduction

Edge computing distributes work onto many small, physically exposed, heterogeneously owned machines. Three facts collide there. The perimeter model fails, because a node *inside* the network can be compromised, rented, misconfigured, or merely selfish, while classical balancers assume every backend is honest about its state and faithfully does its work. Self-reported telemetry is the norm and is trivially gameable: orchestrators schedule on what a node says about itself, so a node that underreports load attracts traffic. And Zero Trust [1] is policy, not implementation, at the network layer — it says *never trust, always verify, continuously*, but not how a control plane verifies infrastructure nodes, as opposed to users, and turns that verdict into a forwarding decision.

The common answer is a *trust score*: a weighted combination of service-derived features — task outcomes, latency, the gap between claimed and observed load — smoothed over time and compared against an isolation threshold [2, 3]. We implemented exactly such a score, and then asked the question an ablation asks: *if we remove everything except the score, what stops working?*

An adversary that misreports its load but completes every task it accepts keeps its outcome-derived features near their maxima. Only the terms with the smallest weights can move against it, so its score settles well above zero. In 15 live runs at our shipped threshold the score of such an adversary never crossed it. Raising the threshold does make the score fire, and a live sweep measures what that costs: the threshold must sit within a few hundredths of where honest servers fall under load, and even then the score is several times slower than an independent gate. Score-based and criteria-based trust algorithms are the two variants NIST SP 800-207 describes [1]; our system is a hybrid of the two, and this paper measures what each part buys.

That adversary, however, turned out to be slow, and its lie did not attract traffic (§V-B): the gate had caught a slow server, not a lie. So we built liars whose lie works and ran them at a load where a lie can pay (§VII-E). Most lies still do not pay: the honesty term penalises a claim that disagrees with the controller's own count, and power-of-two-choices settles each comparison on score. One does. A server can lie in the instantaneous load figure the router ranks on while reporting a truthful busy-time counter, which is the figure the honesty check verifies. That *split claim* wins traffic and evades both rails, and routing on the verified figure closes it.

I-A. Contributions

1. **A live ablation of the two rails** (§VII-A). The SCORE-ONLY arm is the shipping controller with the gate's enforcement switched off and nothing else changed. At the shipped threshold, two of four server attacks are never contained without the gate (0/5 each) and always contained with it (5/5 each).
2. **A live threshold sweep** (§VII-D), registered as a written prediction before it ran. It shows the score *can* isolate both attackers, but only in a band a few hundredths wide above their floor, which one charged timeout also takes an honest server to, and 3.4× to 20× more slowly than the gate.
3. **A liar whose lie works** (§VII-E): 80 further live runs in four batches, each with a prediction registered before it ran. A consistent lie loses traffic; a split claim gains +4.00 [3.47, 4.52] points and is never quarantined; routing on the verified claim removes the gain (+0.01 [−0.28, +0.30]) at no measured honest cost. The batches also exposed, and confirmed the fix for, a false-positive defect in the honesty check that is

invisible at the shipped load.

4. **Honest slowness and a third witness** (§VII-F, §VII-G). Emulated mixed hardware and throttling cause no false quarantines at the shipped load (one of 3.6 s at a 10% throttle, in 1 of 5 runs). The switches' own counters, joined per connection with the clients' reports, name which party lies — a dropping server, a hiding switch, a badmouthing client, or a dropper covered by one colluding switch — in 25 of 25 runs, with no honest party ever blamed.
5. **Attestation beside behaviour** (§VII-H). A server is admitted only on a signed measurement of the code it runs, taken from outside the process. A tampered server was refused in 5 of 5 runs and handed no work; unattested, the same server took 6–25 connections before the gate caught it. A server compromised at run time passed every quote and only behaviour caught it.
6. **Replicas that check the controller** (§VII-I). Ledger replicas that replay the trust arithmetic reject every forged score that reaches them (5 478 records, none falsely), though routing still acts on it. A controller that forges the evidence instead passes them and blinds its own detectors (honest task success 82.6%); only the switch witness exposed it.
7. **A witness that acts** (§VII-J). Acting on the switch witness's verdicts, and holding a blamed server until the witness sees it serve trial connections, raised honest task success under forged evidence from 82.2% to 99.1%, with one false action in 73 acting runs. Building the hold exposed two defects in the probation path that every earlier recording shares, which we measure and correct (§IX).
8. **An explanation** (§V): the score floor an adversary of this class can hold, why the honesty term barely registers the lie, and why the tested attacker is not latency-faithful.
9. **What enforcement buys** (§VII-K, §VII-L). The gate buys containment and the latency tail, not delivery or availability. A per-client meter against a flooding device does improve honest-device latency under a heavy flood, measured over repeated runs with the throttle on and off.
10. **A defect specific to data-plane enforcement** (§VIII-A): quarantine tears down in-flight connections and the resulting timeouts are charged to the server that inherited the flow unless attribution is made explicit.
11. **A retraction** (§VIII-B) of a containment–attribution trade an earlier version of this work claimed.
12. **A reproducible artifact** (§XI): 328 archived live recordings, an aggregator reporting between-run intervals, an offline model of the liar experiment, 1 461 automated tests, and a source tag on every number.

I-B. Scope

This is a prototype-scale study on an emulated network [4]. We claim "demonstrated in emulation", never "proven in production". The adversary model is behavioural and non-adaptive (§III); the ledger is crash-fault tolerant, not Byzantine; the controller is inside the trusted computing base, though §VII-I measures how far the ledger replicas can check it; and the attackers were built by the same team as the detectors. Each is revisited in §IX.

II. Related Work

Zero Trust and trust-aware edge computing NIST SP 800-207 [1] is the doctrine we implement and the gap we claim: it mandates continuous verification but specifies no mechanism for verifying serving infrastructure, and no coupling between a verdict and the forwarding path. Ali *et al.* [3] is the closest prior work, combining Zero Trust, a blockchain record and MEC load balancing; we differ in enforcing the verdict in the data plane and in evaluating by ablation against a control arm rather than against a separate baseline system. Huang *et al.* [2] score trust for edge task placement using a composite of service-derived features — a score-only design of exactly the form our ablation removes the second rail from. TREN [5] applies SDN and Zero Trust to smart-city edge networks. ETSI GS MEC 003 [6] supplies the deployment mapping we adopt: controller → MEC orchestrator, edge servers → MEC hosts.

Edge and SDN security Edge-security surveys list rogue and compromised edge nodes among the central threats of fog and multi-access edge computing [7, 8]. SDN security work treats the controller as both an enforcement point and a target [9, 10]; our controller sits in the trusted computing base for that reason (§III). In practice, Zero Trust verifies infrastructure by remote attestation [11], which proves what software a machine booted; behavioural verification of the kind studied here is complementary, since a correctly booted server can still misbehave. Multivocal reviews of Zero Trust note that academic work has concentrated on architecture [12]; this paper is about one decision inside it.

Misreporting attacks on load balancers Load misreporting against SDN load balancing is an established attack, so far studied at the switch. A compromised switch falsifies its own traffic statistics and draws a chosen share of load through itself [13]; with reconnaissance it can do so while evading threshold-based detectors [14]; and a learned detector can flag such reports on a research testbed, without acting on them [15]. SPHINX checks flow statistics for consistency between the switches along a path [16]. We study the server-side counterpart: the edge server misreports its own load in an application-level self-report, the controller cross-checks the claim against what it observes itself, and the verdict is enforced in the forwarding path.

Gray failure and outlier ejection A server that answers health checks but serves slowly is a *gray failure* [17]; detecting it needs observations from the requester's side [18], and isolating fail-slow components measurably cuts tail latency in production [19]. Production proxies already separate ranking from exclusion: Envoy's least-request balancer uses power-of-two-choices while

a separate outlier-detection rule ejects hosts, on error and success-rate criteria [20]. Our anomaly gate is an exclusion rail of this kind, driven by a controller-side latency and honesty check rather than error counts.

Load balancing under stale information Our first selector used arg max over a composite score and starved the fleet; we replaced it with power-of-two-choices [21], whose lineage is the balanced-allocations result of Azar *et al.* [22]. The two places our system *loses* in simulation are also literature: herd behaviour from stale load information [23] and the result that stale least-loaded performs worse than random [24]. Our control loop polls at 1 s while tasks complete in $\sim$0.1 s, which is that regime. Fairness uses the Jain index [25], reported over two populations for the reason in §VII-K. Little's law [26] appears as a diagnostic: it is how we established that a live run's reported occupancy was fabricated rather than real.

Primitives Device admission uses PRESENT-80 [27], standardised in ISO/IEC 29192-2 [28] and checked against the cipher's published test vectors. We frame it as lightweight-crypto *architecture* appropriate to constrained devices, not as strong security: an 80-bit key is below modern margins. The trust record store is a hash-chained log with a Merkle root [29] per block; replication uses Raft [30], which is crash-fault and not Byzantine tolerant. Weights are tuned online by UCB1 [31] rather than supervised learning, because no labelled data exists at runtime. The circuit-breaker pattern [32] is the ancestor of our probation mechanism.

What is ours The load-balancing and consensus results are established theory we reproduce on a running system. What we claim as ours is the live measurement of what each rail buys, at the shipped threshold and across a threshold sweep (§VII-A, §VII-D), the flood-response result (§VII-L), the data-plane enforcement-feedback defect (§VIII-A), and the retraction (§VIII-B). Separating a veto from a weighted score is not new in itself; the contribution is the measured size of the score's margin and delay against attackers that complete their tasks.

III. Threat Model and Assumptions

III-A. Adversary

The adversary controls a minority of edge servers and clients. A compromised server may misreport its load in the direction that attracts traffic (*load misreporting*; our code and recordings label this attacker `sybil`, although a Sybil attack in the usual sense is one entity presenting many identities [33]); accept tasks and never complete them (*blackhole*); drop a configurable fraction (*grayhole*); or alternate faithful and malicious phases on a period (*on-off*). A compromised client may flood the service, present another device's identity while holding the fleet key (*identity spoof*), or attempt admission with the wrong key. All seven behaviours arm on a delayed onset, so every attack has a clean baseline interval and an attacker is behaviourally identical to an honest node until it arms.

Two consequences shaped the design. A load-misreporting server that still serves never fails a task, so no outcome-derived feature can condemn it; this is the adversary of §V. Our implementation of it burns CPU in the background while claiming to be nearly idle, so it is also slow, which matters for how it is caught (§V-B). And a server *drowning under a client's flood* is not lying about its load — it is telling the truth about being overwhelmed — so the flood detector is keyed on the requesting client, never on the edge node the flood lands on.

Section VII-E adds three liars that serve at honest speed: a *consistent* liar, whose busy-time counter is faked to agree with its claim; a *split-claim* liar, which lies only in its instantaneous load figure; and an *adaptive* liar that reads its peers' claims from the controller and undercuts them while staying inside the honesty threshold, the server-side analogue of the reconnaissance-tuned switch of [14]. Section VII-G adds a switch that misreports its own counters — hiding every response, or covering for one dropping server — and a client that reports successful tasks as timeouts. Section VII-H distinguishes how a server came to misbehave: from a *tampered image*, code changed on disk before it starts, or from a *run-time compromise* of genuine code.

III-B. Out of scope

We do not model, and do not defend against: collusion between edge nodes; an adaptive adversary that observes its own detection and modulates to stay under the thresholds; attacks on the controller or dashboard; data-plane attacks the controller never observes; and Byzantine behaviour inside the controller cluster. A literal replay attack is structurally impossible in our admission protocol — the nonce is popped on first verification — so we did not build a decorative version of an attack the system already defeats.

III-C. Trusted computing base

The SDN controller is fully trusted. This is the standard SDN trust model and it is in tension with the doctrine we implement, so we state the cost: the controller is a single point of compromise and of failure, and Raft replication of ledger state is a step toward redundancy, not a solution. Section VII-I measures how far replicas can check the controller without trusting it: they can recompute every score, but not the evidence it was computed from. The ledger provides tamper-evidence (retroactive falsification of trust records is detectable and localisable to the block where it happened) and availability under replica crashes, within a cluster inside the TCB. It is not a permissionless blockchain and does not tolerate Byzantine replicas. A hash-chained Merkle log replicated by Raft is, in substance, a signed append-only log structured for per-record proofs; we use that description rather than defend a label. When attestation is on (§VII-H), each server's attester process and its signing key are trusted too.

IV. System Architecture

The controller is a single OpenFlow 1.3 [34] application; the trust engine, the authenticator and the ledger are in-process libraries. A client never addresses a server: it connects to a virtual IP and the controller rewrites destination MAC and IP in the data plane,

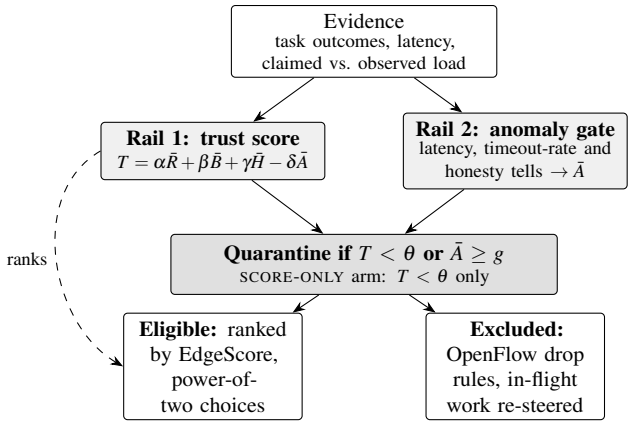


Figure 1. The two decision rails. The trust score *ranks* eligible servers; the anomaly gate *excludes* them, and exclusion is applied before ranking rather than as a score penalty. $\bar{A}$ also enters the score through its $-\delta\bar{A}$ term in every arm. The SCORE-ONLY ablation removes only the gate's ability to act: $T < \theta$ **or** $\bar{A} \geq g$ becomes $T < \theta$.

so the routing decision is invisible to the client and *revocable* by the controller. One design rule runs through every layer and is what makes the ablation cheap: the *request* path writes evidence into the trust state and the 1 Hz *control* loop reads decisions out of it, and the two never share a code path.

IV-A. Rail 1: the trust score

Each completed task yields an update carrying task status, measured end-to-end latency, the CPU load the node *reported*, and the load the controller *observed*. Four features are extracted and smoothed with decay $\lambda = 0.85$:

$$r = \begin{cases} 1.0 & \text{success} \\ 0.3 & \text{timeout} \\ 0.0 & \text{failure} \end{cases} \qquad b = \max\left(0,\, 1 - \tfrac{\ell}{500\,\text{ms}}\right)$$

$$h = \max\left(0,\, 1 - \tfrac{|c_{\text{rep}} - c_{\text{obs}}|}{0.5}\right) \qquad a = \mathbf{1}[\text{anomaly raised}]$$

and the composite is

$$T \;=\; \alpha\bar{R} + \beta\bar{B} + \gamma\bar{H} - \delta\bar{A}, \tag{1}$$

with α=0.35, β=0.25, γ=0.25, δ=0.15, clipped to $[0,1]$ and initialised at 0.5. $\bar{R}$ is reputation from outcomes, $\bar{B}$ behavioural consistency from latency, $\bar{H}$ resource honesty from the claimed-versus-observed gap, and $\bar{A}$ a smoothed suspicion term.

IV-B. Rail 2: the anomaly gate

The second rail is independent of Eq. 1 and its verdict is not a weighted contribution to anything. A 1 Hz loop polls each node and runs four detectors: a **latency tell** (a node claiming to be idle while answering slower than a multiple of the *fleet median*, requiring persistence across three polls); a **timeout-rate tell** over a bounded window; a **CPU-honesty check** comparing claimed busy time against the duty cycle implied by the dispatches the controller saw complete, which *abstains* when the controller has too few completions rather than falling back to a fleet comparison; and a **flood tell** keyed on the requesting client. Firings are smoothed into $\bar{A} \in [0,1]$.

A node is quarantined when $T < \theta = 0.30$ *or* $\bar{A} \geq g = 0.50$ (Fig. 1). The disjunction is the architecture, and the second disjunct is what §VII-A removes.

IV-C. Selection, enforcement, and the exit from quarantine

Exclusion is applied *before* ranking. Among eligible nodes the controller scores $w_1 T + w_2(1 - \text{cpu}) + w_3(1 - \widehat{\ell})$ with $w = (0.50, 0.30, 0.20)$ and selects by power-of-two-choices (d=2, $\varepsilon = 0.05$). If every candidate is ineligible the request is denied — deny by default, never fall back to an untrusted node. Between "full" and "quarantined" sits a rate-limited band implemented with OpenFlow meters.

Once a rule is installed, matching packets are forwarded entirely in the data plane and never return to the controller [34], so all controller-side observation in this paper is derived from 1 Hz flow-statistics counters and endpoint reports rather than from per-packet inspection.

A decision becomes a flow-mod on a real switch: quarantine installs a high-priority drop rule, deletes the node's serving rules by cookie, and re-assigns in-flight dispatches. Quarantine naively implemented is an *absorbing state*, because isolating a node cuts exactly the traffic that produces the outcomes $\bar{R}$ and $\bar{B}$ are built from. We close this with **probation**, the half-open leg of a circuit breaker [32]: a node quarantined on *trust alone*, whose anomaly rail has fallen back below the gate, is offered one trial task every 5 s. A node the gate has flagged is never probed.

IV-D. Naming the attack, admission, and audit

A classifier turns detector signals into one discrete label per subject. It is a pure function over an evidence *window* — not a single cycle — because two attack pairs are not separable from one cycle's signals: blackhole and grayhole raise the same signal and differ in *magnitude*; Sybil and on-off raise the same signals and differ in *intermittency*. The live controller and the offline scorer call one implementation, so they cannot drift.

Devices authenticate by PRESENT-80 challenge–response with single-use nonces. Because the key is fleet-wide the cipher authenticates *possession of the key*, not the *device*; we close that with a source-IP pin against a provisioned roster. A refusal returns HTTP 403 *and* publishes an event, so a denial is visible to the recording — without which a spoof leaves no trace and cannot appear in a confusion matrix. Trust updates are batched ten to a block into a hash-chained ledger with a Merkle root per block, committed to a single in-process ledger or, optionally, to a three-replica Raft cluster (§VII-N); the dashboard re-derives the chain in the browser rather than displaying the controller's own validity flag, because a panel rendering that flag could never contradict it. Optionally, every replica also replays the trust formula from the evidence in each record and appends only the records whose scores it reproduces (§VII-I).

Servers can also be made to earn admission (off by default). An attester process beside each server's agent measures, from outside the agent process, the interpreter and the source files it actually runs, and signs the measurement together with a fresh controller nonce under the server's Ed25519 key. The controller admits a server only while its latest quote matches the reference

build and is younger than 75 s; quotes repeat every 30 s, and any refusal revokes admission at once. An unadmitted server is never selected, not even for a probation trial. Measuring from outside matters: code that reports its own hash can report any hash.

V. What a Score Can Do Against a Load-Misreporting Server

This section states what we expected the experiments to find, and why. It concerns scores of the form of Eq. 1: a convex combination of service-derived features minus a bounded suspicion term.

V-A. A floor under the score

Consider a server that misreports its load in the direction that attracts traffic but completes every task it accepts. Against it the outcome feature is pinned near its maximum, $\bar{R} \to 1$, and the latency feature sits at whatever its own service latency gives, $\bar{B} = b$. Only $\bar{H}$ and $\bar{A}$ can move against it.

Proposition 1 (Score floor). *Let $T = \alpha\bar{R} + \beta\bar{B} + \gamma\bar{H} - \delta\bar{A}$ with $\alpha + \beta + \gamma + \delta = 1$, all weights non-negative and all smoothed features in $[0,1]$. For a server with $\bar{R} = 1$ and $\bar{B} = b$, and for* any *values of the remaining features, $T \geq \alpha + \beta b - \delta$, attained at $\bar{H} = 0$, $\bar{A} = 1$.*

Proof. T is non-decreasing in $\bar{H}$ and non-increasing in $\bar{A}$, so it is minimised at $\bar{H} = 0$, $\bar{A} = 1$, giving $\alpha + \beta b - \delta$. □

The bound is elementary; what matters is what it bounds. In a weighted sum, good service compensates for dishonesty: the penalty available is capped by δ, the weight left over after the features that measure service. Isolating such a server by score therefore needs $\theta > \alpha + \beta b - \delta$. With our weights that is $0.20 + 0.25\,b$: 0.44 for a server as fast as an honest one ($b = 0.96$ at 20 ms), and lower for a slower one. The gate is the opposite kind of rule: a veto, which no amount of good service can offset.

V-B. The tested attacker, and why the bound is loose

Three measured facts qualify the proposition.

The tested attacker is not latency-faithful. Our load-misreporting server burns CPU in the background, so it answers slowly. Computing the floor from each run's measured latency for that server gives 0.302 on average (range 0.278–0.328) across the ten runs that let it keep serving, which corresponds to a mean service latency near 300 ms, far above the fleet's. That slowness is what the gate's latency tell detects, and it is why the 0.44 figure above, which assumes honest latency, does not apply to this attacker. A misreporting server that does *not* slow itself was not tested (§IX).

The honesty term barely registers the lie. The measured minimum score was 0.399–0.409, about 0.10 above the bound, because $\bar{H}$ never approached zero: the claimed-versus-observed load gap averaged only 0.084, so $h = 1 - \text{gap}/0.5$ averaged 0.83 and $\gamma\bar{H}$ added about $+0.10$ throughout. The normaliser assumes a gap the adversary never has to open.

The lie did not attract traffic. The point of misreporting load is to draw work, yet after onset the load-misreporting server received 1.98% [1.59, 2.36] of routing decisions in SCORE-ONLY, against a fair share of 12.5%, and 9.22% even with no enforcement at all. At this workload's occupancy of about 1% an honest server truthfully reports a load of about zero, so the attacker's fixed claim of 0.10 ranks *below* the truth, and its slowness lowers T. The gate then took an attacker the score had already starved from 1.98% to 0.45%; the on-off attacker, which kept 9.53% under the score alone, is where the gate's containment mattered (5.04% with it). Whether the gate holds against a liar whose lie works is the question of §VII-E. [evidence/replicates/summary.txt]

So the proposition says a threshold low enough to be safe for honest servers leaves a floor the attacker can sit above; it does not say how high a threshold would have to go in practice, or what that would cost honest servers. §VII-D measures both.

Table 1. The arms. Each of the first three differs from the one above it by one mechanism; the sweep and flood arms differ from their parent by the settings shown.

Arm	Enforcement
CONTROL	none. Static nearest-server routing; detectors run and publish, nothing acts on them.
SCORE-ONLY	trust rail only. Quarantine, rate-limit and probation eligibility decided by $T < \theta$ alone.
BOTH-RAILS	both rails. $T < \theta$ **or** $\bar{A} \geq g$.
sweep	SCORE-ONLY with $\theta \in \{0.40, 0.45, 0.50\}$ in place of 0.30.
flood off/on	BOTH-RAILS with a heavier flood; *on* adds the per-client flood meter of §VII-L.

VI. Evaluation Methodology

VI-A. Testbed and workload

One core switch and eight edge switches, all Open vSwitch speaking OpenFlow 1.3 under Mininet [4], carrying eight edge servers and forty IoT client processes on a four-core host. Servers perform *real bounded work* — hash loops sized by a configured budget — rather than sleeping, which is what makes the telemetry measured rather than injected. Each device reports every 2.0 s with a 15 ms task budget and a 4.0 s client timeout: ∼20 requests/s from honest devices plus the flooding device (three workers paced at 0.25 s; twelve at 0.1 s in the heavy-flood arms of §VII-L). The rate is chosen so the *control plane*, which is what is under test, is measured rather than starved by the host.

Six attacks plus wrong-credential devices arm on a staggered schedule inside every run: srv3 load misreporting at 20 s, srv6 blackhole at 30 s, srv1 grayhole (0.5 drop rate) at 40 s, srv8 on-off (20 s period, 0.5 duty) at 50 s, iot37 flood at 60 s, iot38 identity spoof at 15 s, and iot39–40 with wrong keys. Four servers and thirty-six devices are honest and must not be flagged.

VI-B. The arms

All three run the same topology, the same forty devices, the same six attacks on the same schedule, launched by the same code, with trust computed by the same estimators. CONTROL is the no-enforcement control. SCORE-ONLY is the ablation that matters: it is the *shipping controller* with one boolean changed, and its configuration differs from BOTH-RAILS's in exactly two

keys — the enforcement switch and the recording path — which a test pins.

One subtlety decided the implementation. Setting the gate threshold to infinity would have disabled the gate but *also* removed the $\bar{A}$ term from Eq. 1, silently changing the score family and making the arms incomparable. Instead the anomaly rail has a separate *enforcement* switch: $\bar{A}$ still enters the score in both arms, and only its ability to *act* is removed. Every treatment arm also runs the same online UCB1 weight optimizer [31], which chooses among five EdgeScore weight presets; it never changes the trust score, the thresholds or eligibility, so it moves ranking among eligible servers equally in every arm. Rows the CONTROL arm cannot produce are reported as *not measured*, never as zero — an absent mechanism is not a mechanism that scored zero.

VI-C. Replication and statistical treatment

The unit of replication is the run. Five runs per arm, each $\sim$320 s, executed in rotating arm order so machine drift does not land on one arm: fifteen for the ablation, fifteen for the threshold sweep and ten for the flood experiment. The sweep's outcome was predicted in writing, from the ablation's recordings, before any sweep run existed. Per-task or per-route observations inside a run are not independent replicates and are never pooled across runs.

Continuous metrics are reported as mean with a two-sided 95% Student-t interval over runs. Yes/no outcomes — *was this attacker isolated at all*, *did its score ever cross* θ — are reported as k/n with a Wilson interval [35], never as a mean of 0/1s with a t-interval, which is wrong at this n. Times are reported only over the runs where the event happened, with k/n alongside, because a run where it never happened has no time to average and substituting the run length would invent one.

With $n = 5$ per arm we report intervals and do not run significance tests between arms on the service metrics; we describe two arms as separated only when their intervals do not overlap, and say so explicitly when they do.

VI-D. The score rail is measured in every arm

One measurement makes the ablation interpretable: for every attacker, in *all three arms including the ones that never act on it*, we record whether and when T first falls below θ. This is the score rail's verdict in isolation, observed even where it is not enforced, and it is the direct test of §V. We also record, at each first quarantine, which rail could have caused it: $T < \theta$ only, $\bar{A} \geq g$ only, or both. An arm that does not enforce the gate is always attributed to the score, whatever $\bar{A}$ reads, because its gate cannot act.

VI-E. The liar experiment

A lie about load can only pay if it undercuts what honest servers truthfully claim. At the workload above, honest servers are about 1% occupied and report a load of about zero, so no lie can undercut them. The liar experiment therefore raises *occupancy*, the quantity the load claim and the honesty check are about, to 0.40 without loading the host: servers hold a worker for 320 ms per task instead of hashing, with two workers each, at the same 20 tasks/s. Every other attack is removed; one liar arms on srv3 at 20 s. The arms are an honest-only control and the three liars of §III, with the gate enforced; a "slots" variant also caps each server at two tasks in service. Each pair of arms differs only in the liar, the routing claim and the recording path, pinned by a test.

The experiment ran in four batches, each scored against a written prediction committed before it ran: the first on the shipped detector, the second and third after fixes the previous batch had motivated (§VII-E), and a fourth at the shipped workload. Before any of them, a discrete-event model driving the real selector and trust calculator predicted every arm; its route shares landed within about a point of the live values. Under power-of-d-choices with ε-exploration, a liar that wins every comparison it is sampled into is capped at $\varepsilon/N + (1-\varepsilon)d/N$ of the traffic, 24.4% here against a fair 12.5%, however large the lie.

VI-F. Five further experiments

Five further experiments use the shipped workload with every other attack removed. In the first, Linux cgroup CPU caps emulate mixed hardware: four servers at full speed and four at half a core, and in two arms an honest server throttled to 25% or 10% of a core for 60 s. Tasks are a fixed amount of work (45 000 hash iterations, $\sim$15 ms at full speed), so a capped server really is slower, including in its answers to the controller's polls. Its six arms vary the latency tell's baseline (fleet median or per-class median), the throttle, and where the slow load-misreporting server sits (among slow or among fast servers). In the second, the controller joins every connection's switch counters with the client's report (§VII-G) and publishes who it judges to be lying, without acting on it. A lying switch is emulated where the controller receives its statistics, which is where a compromised switch agent would speak; Open vSwitch itself cannot be made to lie without patching it. Five arms: no liar, two dropping servers, a hiding switch, two badmouthing clients, and a dropper covered by a colluding switch. The witness experiment ran twice, before and after fixes its first batch motivated, each against its own registered prediction. The third crosses attestation (off, on) with how a dropping server came to drop: no attacker; a *tampered image*, the released agent with one line appended, launched from its own directory; or the *genuine image* told to drop at run time. The tampered and run-time servers behave identically and differ only in what is on disk. Five arms, five runs each, against one registered prediction. The fourth commits the ledger to a three-replica Raft cluster and makes the controller dishonest: a test-only switch, which refuses to start without an environment variable, forges one server's trust records from 60 s, either by inflating its score or by rewriting its outcomes as fast successes before the formula runs. Four arms, five runs each: replicas that do not verify, replicas that verify an honest controller, and verifying replicas against each forgery; in the second forgery the server really drops every task. The fifth lets the witness's verdicts act (§VII-J) against that evidence forgery, against two badmouthing clients, and against the two dropping servers, each acting arm beside an observe-only twin, and adds a *hold* variant. It ran in four batches, each against its own registered prediction:

Table 2. The ablation at $\theta = 0.30$. *Score crossed* θ is the trust rail's own verdict, recorded in every arm including those that never act on it. *Quarantined* is what the arm actually did. Proportions carry Wilson 95% intervals; $\min T$ is the mean over runs of the lowest score reached after onset.

	CONTROL	SCORE-ONLY	BOTH-RAILS
srv6 — blackhole (fails every task)			
score crossed θ	5/5	5/5	5/5
quarantined	—	**5/5**	**5/5**
$\min T$	0.000	0.148	0.130
srv1 — grayhole (fails half its tasks)			
score crossed θ	5/5	5/5	5/5
quarantined	—	**5/5**	**5/5**
$\min T$	0.000	0.180	0.129
*srv3 — load misreporting (**never fails a task**)*			
score crossed θ	**0/5**	**0/5**	**0/5**
quarantined	—	**0/5**	**5/5**
$\min T$	0.399	0.404	0.617
*srv8 — on-off (**never fails a task**)*			
score crossed θ	**0/5**	**0/5**	**0/5**
quarantined	—	**0/5**	**5/5**
$\min T$	0.411	0.399	0.548

$\theta = 0.30$. Wilson 95% interval for 5/5 is $[57\%, 100\%]$ and for 0/5 is $[0\%, 43\%]$; the two do not overlap. "—" is *not measured*: CONTROL has no quarantine mechanism.

one acting, then three of the hold, the second and third after fixes to the trial path that the first two exposed.

VI-G. Simulation levels

Two results in §VII-O come from a seeded discrete-event simulation driving the *real* selector, trust calculator and both shipped detectors: a comparison against four routing baselines over 600 runs, and a selector-scalability sweep to $N = 64$ that the four-core host cannot run live. These are labelled as simulation wherever they appear and no live claim rests on them.

VII. RESULTS

All live figures come from fifteen runs archived under `data/runs/`, scored by one aggregator whose output ships with this draft. [evidence/replicates/summary.txt]

VII-A. The ablation at the shipped threshold

Table 2 divides the four server attacks into two groups along the line §V predicts.

The two attackers that **fail tasks** — the blackhole and the grayhole — drive $\bar{R}$ down, and their scores cross θ in every one of fifteen runs. The score rail isolates them unaided: in SCORE-ONLY, both are quarantined 5/5.

The two attackers that **never fail a task**, the load-misreporting server and the on-off attacker, are a different matter. At $\theta = 0.30$ their scores **never cross it**, in any arm, in any run: 0/15 each. Their minimum trust over a 320 s run settles at 0.399–0.411 in the arms that let them keep serving, comfortably above the threshold and consistent with §V-B. With only the score rail enforced, neither is *ever* quarantined (0/5 each, Wilson $[0\%, 43\%]$). With the gate enabled, both are quarantined in *every* run (5/5 each, Wilson $[57\%, 100\%]$).

Table 3. At each attacker's first quarantine in BOTH-RAILS: the rail that could have caused it, and T at that instant (mean over 5 runs, 95% t-interval).

Node	Attack	Rail	T at quarantine
srv6	blackhole	score 4/5, both 1/5	0.130 [0.074, 0.185]
srv1	grayhole	both 3/5, gate 2/5	0.220 [0.066, 0.374]
srv3	load misreport.	**gate 5/5**	**0.710** [0.630, 0.791]
srv8	on-off	**gate 5/5**	**0.709** [0.641, 0.777]

"both" means $T < \theta$ *and* $\bar{A} \geq g$ held at that instant, so the score would have isolated the node unaided. Crediting the gate in that case would overstate it.

Table 4. Seconds from attack onset to isolation. Median over 5 runs, with the range. CONTROL never contains anything.

Node	Attack	SCORE-ONLY	BOTH-RAILS
srv6	blackhole	10.7 (9.8–12.0)	10.1 (9.3–11.7)
srv1	grayhole	15.6 (12.1–22.6)	15.5 (12.3–31.7)
srv3	load misreport.	**never**	**7.6** (5.9–10.2)
srv8	on-off	**never**	**10.4** (9.8–37.5)

At the shipped threshold, then, the gate is the only mechanism that contains half of the server attacks. Whether a different threshold would let the score do it alone is the question §VII-D answers.

VII-B. Which rail fired, and at what score

Table 3 shows the same split from the enforcement side. The load-misreporting server and the on-off attacker are quarantined at a trust score of 0.71 — 2.4× the isolation threshold — by a rail that is not part of the score. The blackhole is isolated by the score in four of five runs, with the gate merely concurring in the fifth. The grayhole sits between: both rails qualify in three runs, the gate alone in two, which is the case where the gate is simply faster.

We flag one measurement trap here because it changed a number in this table. Attributing the rail from "was $\bar{A} \geq g$ at that instant" is wrong in the SCORE-ONLY arm, where $\bar{A}$ still rises but cannot act: every grayhole quarantine there carries $\bar{A} \approx 1.0$ next to $T < \theta$, and a naive reading credits a gate that was switched off. Attribution therefore consults the arm's enforcement setting, and reports "both" rather than "gate" when both conditions hold.

VII-C. Containment latency

Where both arms contain an attacker they contain it at the same speed. The gate is also the fastest single detector in the set: it reaches the load-misreporting server in a median 7.6 s, faster than the score reaches either task-failing attacker, because a latency tell needs three consecutive polls whereas a timeout rate needs enough dispatches to accumulate.

Detection and response are effectively simultaneous: over the twenty node-run pairs in the BOTH-RAILS arm the gap between a node's first anomaly event and its first quarantine ranged from 3.5 to 12.5 ms, so the latencies above are detection latencies. [data/runs/treat/]

VII-D. The threshold sweep: a narrow, slow band

The obvious objection to Table 2 is that $\theta = 0.30$ was our choice. Before running anything we wrote down a prediction from the

Table 5. The score rail alone at four thresholds, against the gate. Runs out of 5 in which each node was isolated after its attack began; in brackets, the median time from onset to isolation. Five runs per row.

Arm	srv3 load mis.	srv8 on-off	Honest q.	$p95$
score, θ=0.30	0/5	0/5	0/5	176 ms
score, θ=0.40	0/5	1/5 (229 s)	0/5	173 ms
score, θ=0.45	**5/5** (26 s)	**5/5** (207 s)	0/5	157 ms
score, θ=0.50	5/5 (50 s)	5/5 (76 s)	**1/5**†	149 ms
both rails, θ=0.30	5/5 (7.6 s)	5/5 (10.4 s)	0/5	137 ms

†Plus one attacker quarantined before its attack began. Wilson 95% intervals: 5/5 [57%, 100%], 1/5 [4%, 62%], 0/5 [0%, 43%].
[evidence/theta_sweep/summary.txt]

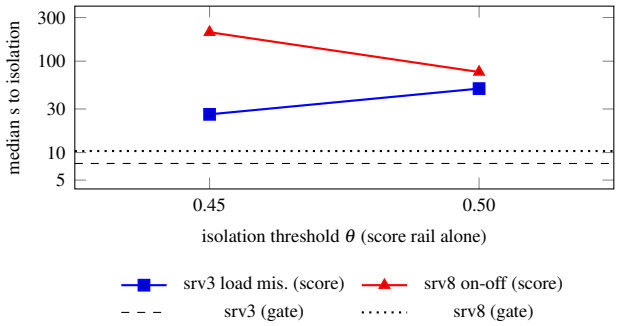


Figure 2. Time from attack onset to isolation. At $\theta \leq 0.40$ the score isolates neither attacker in most runs (Table 5); where it does, it is 3.4× to 20× slower than the gate at the shipped threshold (horizontal lines).

ablation's own recordings: at $\theta = 0.45$ the score would cross for both attackers in every run with no honest server affected, and at 0.50 an honest server would dip below once. We then ran the score-only arm live at three higher thresholds, five runs each (Table 5, Fig. 2). The prediction held at 0.45 and 0.50; at 0.40 it expected the on-off attacker to be caught in four runs and it was caught in one.

The score is not blind. At $\theta = 0.45$ it isolated both attackers in every run with no honest server quarantined. Three things limit what that buys.

It is slow. Median time to isolation was 26 s for the load-misreporting server and 207 s for the on-off attacker, against 7.6 s and 10.4 s for the gate: 3.4× and 20× slower.

The band is narrow. While they keep serving, the two attackers' scores bottom out at about 0.40–0.41, so a threshold must sit above that to fire. Honest servers fell to 0.417 and 0.420 in two of the $\theta = 0.40$ runs and to 0.421 in one both-rails run, each time from a single 4 s timeout charged to them; under $\lambda = 0.85$ one such timeout takes a steady honest server to about 0.42. Those three timeouts came from a defect in the probation path (§IX), but a real timeout does the same. The usable gap between the attackers' floor and where one timeout puts an honest server is a few hundredths at best. $\theta = 0.45$ held in its five runs, but a run with an honest dip like those would have quarantined an honest server.

The ceiling is close. Every server starts at $T = 0.5$, so no threshold can go much higher, and at $\theta = 0.50$ an honest server was quarantined in one run and an attacker before its attack began in another.

Table 6. Liars that serve at honest speed, at 0.40 occupancy unless marked (5 runs per row). Share of routing decisions after onset with 95% interval; fair share 12.5%. "Raw" ranks on the instantaneous load claim, "verified" on the busy-time duty the honesty check validates.

Liar	Routing, fixes	Share	Quar.	Median
Consistent	raw, shipped	9.37% [9.05, 9.68]	5/5	8.2 s
Consistent	raw, fix 1+3	10.29% [9.99, 10.60]	5/5	6.4 s
Adaptive	raw, fix 1	11.34% [10.97, 11.71]	5/5	17.7 s
Split claim	raw, shipped	17.01% [16.16, 17.86]	4/5†	51.7 s
Split claim	**raw, fix 1**	**16.50% [15.97, 17.02]**	**0/5**	—
Split claim	**verified**, fix 1	12.51% [12.22, 12.80]	3/5‡	—
Consistent*	raw, fix 1+3	10.14% [8.92, 11.36]	0/5	—
Consistent*	verified, fix 1+3	13.58% [11.97, 15.19]	0/5	—

*At the shipped 1% occupancy. †Every firing was the defect described below; none survives its fix. ‡Brief: the liar was isolated 0.34% of the time.

The gate, by contrast, isolated both attackers in every run while their trust was about 0.71: its decision does not depend on where the threshold sits. That is the measured difference between a score-based and a criteria-based trust algorithm in this setting: a margin of a few hundredths and a delay of tens to hundreds of seconds, against a wide margin and a delay of seconds.

VII-E. A liar whose lie works

Table 6 gives each liar's share of routing decisions after onset, against a fair 12.5%, over the four batches.

A consistent lie does not pay. A liar whose busy-time counter agrees with its claim of zero received 9.4–10.5% of the traffic, less than its fair share, and the gate contained it in every run within a median of 6–8 s. Its counter is what betrays it: claimed busy time per completed task falls below the fleet median, the honesty term costs it a little trust on every task, and power-of-two-choices decides each comparison on score, so it loses comparisons it would otherwise tie. The adaptive liar, which undercuts its peers only as far as the honesty threshold allows, fared no better.

The split claim pays, and nothing catches it. A liar that reports a load of zero but a truthful busy-time counter gave the honesty check nothing to find, because the check compares the counter, not the claim, with the controller's own completion count, while the router ranked on the claim. It received 16.5% of the traffic, +4.00 [+3.47, +4.52] points above its fair share, a third more work than an honest server, and was never quarantined. It cost honest service nothing measurable (honest-device mean latency 397.5 against 396.4 ms in the control), so nothing an operator watches would show it. *Ranking on the verified figure closes it*: with the router reading the same busy-time duty the honesty check validates, the split claim's excess fell to +0.01 [−0.28, +0.30] points, with honest latency unchanged (396.2 against 396.4 ms).

The experiment found its own detector's defect. In the first batch every arm, the control included, quarantined 3.0–4.4 honest servers per run, all through the honesty check. The controller's completion rate divided the completions in its window by the time up to the *last* completion, not up to now, so a server that finished a burst and then went quiet read as roughly three

times its real rate. Rebuilt at each of the 119 firings, the expected duty was 0.81 as computed and 0.30 with the span ending now; the server had been quiet a median of 2.0 s. A second, smaller defect let the check judge a partial window during the synchronized start of all forty clients. With both fixed, honest false quarantines fell from 3.4 to 1.4 to 0.2 per run. Both defects are latent at the shipped load, where even a tripled expected duty stays under the threshold; no ablation or sweep run recorded an honesty firing on an honest server, so the results above do not depend on them. Every split-claim quarantine in the first batch was this defect, not detection.

At the shipped load the fix costs nothing measurable. With honest servers about 1% occupied, verified ranking gave a consistent liar more traffic than raw ranking did (13.58 against 10.14%, disjoint intervals) but not measurably more than its fair share ($+1.08$ [-0.53, $+2.69$] points), and no honest server was quarantined. Under raw ranking its claim of zero only ties an honest server's truthful zero and loses the tie on its honesty deficit; under verified ranking it undercuts an honest duty of about 0.01. By a decision rule registered before that batch ran, verified ranking is now the controller's default. [evidence/liar/]

VII-F. Honest slowness is not mistaken for lying

A gate that isolates a server for answering slowly risks isolating an honest slow one: older hardware, or a box throttling in the heat. At the shipped load it does not. With half the fleet at half a core, no honest server was quarantined in any run, with the latency baseline taken over the fleet or over each hardware class (0 in 5/5 runs each), nor with an honest server throttled to 25% (0/5). The slow load-misreporting server was still contained in every run among slow servers (median 9.0 s) as among fast ones (8.4 s).

Throttled to 10%, an honest server's task latency rose from about 90 to over 210 ms, and it was quarantined in one run of five, for 3.6 s in total. The latency tell never fired: its three-poll persistence absorbed the isolated slow replies, and the selector itself routed less work to the slow server. The quarantine came from two other paths: a single `/status` poll that exceeded the 0.5 s timeout, which the unreachable signal counts without persistence, and then the honesty check firing in reverse, because the quarantine had stopped the completions from which the controller's expected duty is computed (§VIII-A). [evidence/hw/]

VII-G. Who is lying: the switches as a third witness

The controller holds three accounts of every task: the server's self-report, the client's report, and, kept by every switch, the packet and byte counters of the connection's two VIP rules. Offline, over 110 recordings, the third agrees with the second to a median of 0.9992 per honest server-run: an honest task is always six packets and 548 bytes in, six and 598 back. Per connection the witness compares whether a full response passed the switch with whether the client reported success. Agreement that the task failed points at the server; a client reporting failure for a response the switch saw points at the client; a client reporting success for a response the switch did not see points at the switch. A client only ever uses its own ingress switch, so a lying switch looks like all its clients lying: the switch is blamed when most of its clients disagree with it, a client alone only when its switch-mates are clean, and a server only on connections whose client and switch are unblamed. Connections the controller re-steered are never judged.

Table 7. The switch witness (observe-only), 5 runs per row, before and after the fixes its first batch motivated. Blamed $k/5$, median seconds after the liar's onset.

Who lies	First batch	After fixes
Nobody	none blamed	none blamed
Blackhole, grayhole	4/5, 5/5 (54, 61 s)	**5/5, 5/5 (13, 14 s)**
Hiding switch	5/5 (14 s); clients*	**5/5 (5 s)**
Two badmouthing clients	5/5, 5/5 (13, 14 s)	**5/5, 5/5 (12 s)**
Blackhole + covering switch	5/5 (59 s)	**5/5 (16 s)**
Honest parties blamed	in 3 of 25 runs, 1–4 s	**never, in 25 runs**

*While the switch's lie ramped up, one or two of its clients were blamed for 1–4 s in 3 of 5 runs before the switch was.

The first batch found two flaws in our design (Table 7). Judged over a 60 s window, a dropping server's honest history diluted its failures while the gate's quarantine starved new ones, so the witness took about a minute; and a lying switch's clients crossed the threshold one by one, before the switch did. Judging each party on its most recent connections, and blaming a client alone only when its switch-mates are clean, fixed both; each fix is pinned by a test that reproduces the live failure. In the re-run every liar was blamed in every run and no honest server, switch or client was blamed at any time. The covering switch itself was never blamed: the gate isolates the dropper within seconds, leaving too few covered connections to judge, and covering a server the gate isolates anyway gains the switch nothing. The witness only observes; letting its verdicts act, for example by discounting a blamed client's reports, is a separate experiment. [evidence/witness/]

VII-H. What attestation adds to behaviour

Table 8 runs the same dropping server three ways. With attestation off, a tampered server is caught by behaviour alone, after it has been handed 6 to 25 connections. With attestation on, the same server is refused in every run, each refusal naming the one file that differs from the reference, and is handed nothing; honest devices see no timeouts at all, against 24.6 per run without it, with disjoint intervals. A server whose genuine code is compromised at run time passes all 42 of its quotes after the compromise, and behaviour catches it as fast as the tampered one. Neither mechanism replaces the other: attestation stops what is wrong before it runs, behaviour stops what goes wrong while it runs.

The cost is small. With no attacker, all eight servers were admitted about 4 s after the controller started (4.1 [3.9, 4.2] s), when clients begin sending anyway; 0 to 3 connections per run were denied before then and none after. Verifying a quote takes 0.2 ms, and no genuine server was refused or lost admission in any of the 15 attestation-on runs. Honest mean latency rose by 2.1 ms (90.6 [88.8, 92.3] against 88.5 [87.2, 89.7] ms); the intervals overlap, but only just, so a small cost is not ruled out. Staggered admission has one side effect: in one run of fifteen

Table 8. The same dropping server, run three ways (5 runs each). Connections: handed to it between its onset and its first quarantine. Mean [95% CI] or median (range).

	Tampered, attest. off	Tampered, attest. on	Run-time, attest. on
Admitted	—	0/5	5/5
Connections	17 [8, 25]	**0**	20 [15, 24]
Contained (s)	10.9 (8.8–32.3)	never served	9.9 (8.9–10.5)
Honest timeouts	24.6 [16.4, 32.8]	**0.0**	27.6 [22.8, 32.4]
Honest success, %	99.52 [99.36, 99.69]	**99.98** [99.95, 100]	99.48 [99.39, 99.57]

Table 9. Ledger replicas against an honest and a forging controller (5 runs each). Forged: srv3's records from 60 s. Share: srv3's share of routing after 60 s (about 13% before). Mean [95% CI] or pooled count.

Controller	Forged rejected	Others rejected	Share	Honest success, %	Commit ms
Honest, not verified	—	0	—	99.97 [99.94, 100]	9.5
Honest	—	0	—	99.96 [99.95, 99.98]	9.7
Forges scores	5 478/5 482*	0	23.9%	99.98 [99.95, 100]	9.5
Forges evidence	0/3 787	0	23.6%	**82.58** [82.06, 83.09]	9.7

*The four not rejected were written in the last second of two runs and never committed. Commit: median per block.

the first server admitted, alone for 56 ms, took 28 of the first 40 connections and its trust fell to 0.354, just above the threshold. [evidence/attest/]

VII-I. Replicas that check the controller's arithmetic

The controller is trusted, but its ledger need not take its word. Every trust record carries the evidence its score was computed from and the weights are fixed, so a replica holding the same trust configuration can replay the formula and check each record's score before and after; records that fail are kept out of the block and reported back. The replay advances on every record's evidence, so one forged score costs at most two records, and the replicas agree because they apply the same log in the same order. A batch the replicas refuse whole is now dropped from the controller's queue; before, it would have been retried forever and blocked every later batch.

Table 9 makes the controller raise one honest server's trust by 0.3 from 60 s. Every forged record that reached the replicas was rejected, and none of the other 22 869 was. Verification added 0.2 ms to a 9.5 ms commit, and a rejection reached the controller in a median 0.22 s. It did not stop the controller: routing kept using the inflated score, and the favoured server's share of traffic rose from 13% to 24%, the power-of-two-choices ceiling for one server. The ledger proves the lie; it does not prevent it.

A controller that forges evidence instead, rewriting a dropping server's failures as fast successes, writes records whose arithmetic is honest, and the replicas rejected none of them. The same forgery blinded the controller's own detectors, which read the same outcomes: the dropping server was never quarantined, kept a trust score near 0.84, drew 24% of all traffic, and honest-device task success fell to 82.6%. Only the switch witness, which reads the clients' reports and the switches' counters directly, saw it: it blamed that server in every run, a median 11.1 s after it began dropping, and blamed no honest party in any of the 20 runs. Because the witness only observes, the loss continued; §VII-J lets it act. [evidence/ledger/]

Table 10. The witness acting and holding (5 runs per row). Quarantined: share of the dropper's time after its onset; connections: handed to it after its onset. Two values: blackhole / grayhole. Mean [95% CI].

Dropper	Witness	Quarantined, %	Connections	Honest success, %
Forged evidence	observes*	0	770 [766, 774]	82.24 [82.07, 82.42]
	acts	84.0 [82.6, 85.5]	135 [122, 147]	97.68 [97.49, 97.87]
	holds	95.3 [95.1, 95.6]	64 [58, 70]	**99.06** [98.96, 99.16]
Blackhole + grayhole	acts	54 / 76	32 / 62	99.11 [98.88, 99.35]
	holds	96 / 78	40 / 60	98.91 [98.81, 99.00]
None	holds	—	—	99.99 [99.97, 100]

*From the first acting batch; the other rows are the final hold batch, on the corrected trial path.

VII-J. When the witness acts

Two actions, both off by default, let the witness's verdicts change what the controller does. A server the witness blames raises an anomaly-gate signal, so quarantine and release follow the gate's normal path; a client it blames has its reports published, so availability still counts them, but charged to no server. Neither reads the controller's task records, which is why the first still works when those records are forged. Each acting arm has an observe-only twin, and every batch ran against a prediction registered first.

Against the controller of §VII-I that forges a dropping server's evidence, acting restores what the controller's own detectors lost (Table 10). The dropper, never quarantined while the witness only observed, was quarantined in every run a median 11.0 s after its onset, and honest task success rose from 82.2% to 97.7%. The rest of the loss is a cycle: the verdict lapses once the dropper's failed connections age out of the witness's 60 s window, the gate releases it, its forged trust draws traffic back, and the witness needs about ten fresh failures to blame it again.

A *hold* closes the cycle. A server the witness blames stays quarantined until the witness itself sees it serve three trial connections in a row, offered one every 10 s. The trial ignores trust and $\bar{A}$, which a forging controller holds high and the gate holds at its ceiling, and the witness judges only the trial connections, so neither the controller's records nor the server's old failures decide the release. Holding raised honest task success to 99.1%, closing 95% of the gap between the observing witness and an honest run, against 87% for acting alone; the forging dropper failed all 23 of its trials in every run and was never released. Against droppers the gate already contains it buys little: the blackhole, failing all of its trials, took 40 connections against 32, each trial a lost task for an honest client, and honest success moved from 99.11% to 98.91% with intervals that overlap. The grayhole, dropping half its tasks, passed three trials in a row and was released once or twice in every run, then blamed and held again. That is the rule's registered weakness: at a 50% drop rate, three successes in a row take fourteen trials on average.

Discounting a badmouthing client's reports cut the reports charged against honest servers from about 259 to about 10 per run, and the share of the time honest servers spent below trust 0.45 from 19.7% to 0.6%. It did not hold the lowest honest trust above 0.45, as we had predicted: under $\lambda = 0.85$ one charged 4 s timeout takes a steady honest server to about 0.42, and the ten or so reports charged before the witness has seen enough

Table 11. Service and containment across the three arms. Mean over 5 runs with a two-sided 95% Student-*t* interval. Bold marks an arm whose interval does not overlap either of the others; where intervals overlap we draw no conclusion. "—" is *not measured*.

Metric	CONTROL	SCORE-ONLY	BOTH-RAILS	**Reading**
Packet delivery ratio	94.35 % [94.15, 94.55]	99.40 % [99.29, 99.52]	99.25 % [99.06, 99.44]	enforcement helps; rail choice does not
Tasks lost to timeout	377.8 [375.0, 380.6]	35.4 [26.8, 44.0]	43.2 [31.0, 55.4]	enforcement helps; rail choice does not
Throughput (task/s)	20.15 [19.44, 20.86]	19.53 [19.23, 19.84]	19.72 [19.12, 20.31]	all three overlap
Mean task latency	87.4 ms [80.0, 94.7]	98.3 ms [93.7, 103.0]	91.6 ms [86.8, 96.5]	all three overlap
*p*95 task latency	317.8 ms [288.2, 347.3]	175.6 ms [161.1, 190.2]	**137.2 ms** [128.5, 145.9]	the gate halves the tail again
Honest per-device availability	85.72 % [85.63, 85.82]	99.29 % [99.15, 99.43]	99.13 % [98.92, 99.34]	enforcement helps; rail choice does not
Honest devices served $< 50\%$	5.0 [5.0, 5.0]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	enforcement helps; rail choice does not
Jain fairness, honest servers	0.676 [0.643, 0.709]	0.996 [0.994, 0.998]	0.996 [0.991, 1.000]	enforcement helps; rail choice does not
Routes reaching attackers	29.84 % [28.63, 31.05]	17.37 % [16.74, 18.00]	**11.63 %** [10.84, 12.42]	**the gate's main service effect**
Mean attacker time isolated	—	41.38 % [40.21, 42.55]	**67.13 %** [64.28, 69.97]	**the gate's main effect**
Honest servers ever quarantined	0.0	0.0	0.0	no false isolation in any arm
Honest subjects mislabelled	0.0	0.0	0.0	no false accusation in any arm
Attackers quarantined pre-onset	0.0	0.0	0.0	no containment credit is spurious

already do it. A minimum measures the first charged report, not how long an attack goes on.

The first two hold batches exposed two defects in the trial path, which also affected probation in every earlier recording (§IX); the hold rows of Table 10 come from the third batch, on the corrected path, in which all 245 trials were judged and none was charged to another server. In the 73 acting runs of the 83 the witness took one false action. In one run the whole host stalled for one to two seconds: the controller logged nothing, and every server's tasks took 0.5–1.0 s instead of 85 ms. When it recovered, routing on stale load sent sixteen connections to one honest server in 1.5 s, its agent's accept queue of five refused six of them, and six failures were 30% of its recent connections. The witness blamed it, and it stayed quarantined for the last 26 s of the run, though its trust never fell below 0.60. The failures were real, and the switches saw them correctly; what the rule lacks is persistence across windows, the same gap as the unreachable signal (§X). The trust rail came closer than the witness to a false quarantine in that stall: one failed task took another honest server from 0.80 to 0.33, just above θ. `[evidence/witact/, evidence/withold/]`

VII-K. Service, availability and fairness

Table 11 separates three questions that a single-baseline comparison conflates.

Does enforcement help? Unambiguously. Against the no-enforcement control, either acting arm raises delivery ratio by about five points, cuts timeouts by an order of magnitude, lifts honest per-device availability by about thirteen points, removes all five starved devices, and lifts fairness across honest servers from 0.68 to 0.996. Every one of those intervals is disjoint from the control's.

Does the *choice of rail* help those service metrics? No, and this is the result a single-baseline design would have missed. SCORE-ONLY and BOTH-RAILS overlap on delivery ratio, timeouts, throughput, mean latency, honest availability and fairness. On the evidence here the gate buys no service improvement over the score rail alone.

What does the gate buy? Containment. The share of traffic reaching attacker nodes falls from 17.37% to 11.63% and mean attacker isolation time rises from 41% to 67%, both with disjoint intervals — because two of the four attackers are never isolated at all without it. The tail follows: *p*95 latency falls from 176 ms to 137 ms, also disjoint. A faithful liar degrades the service it accepts without ever failing a task, so removing it from the rotation shows up in the tail rather than in the delivery ratio.

Two corrections to our own earlier reporting belong here. First, we previously reported a mean-latency *cost* for enforcement from a single run pair; across replicates the three arms' mean-latency intervals all overlap and we withdraw the claim. Second, we previously reported a throughput cost; those intervals overlap too.

Fairness needs two populations Jain's index over the *whole* roster barely separates the arms (0.605/0.681/0.622) and read alone would say the load balancer made no difference. Both extremes are uneven for opposite reasons: the control arm because the flooding device is statically bound to one server that nothing can move it off, the acting arms because they deliberately starve attackers. *Starving an attacker is correct behaviour and must not be scored as unfairness.* Over the four honest servers, where the metric means what it is for, the arms separate hard. We report both populations plus the attacker share, and recommend the same for any trust-aware balancer evaluation.

VII-L. Data-plane response to a flooding client

Detecting a flood is not the same as stopping it. When the flood tell fires, the controller installs a rule on every switch that matches only that client's *new* connections to the virtual IP, above the rule that punts new connections to the controller and below the per-connection rules of connections already admitted. In the default mode those new connections pass through a per-client OpenFlow meter set to twice an honest device's expected rate, so a misjudged client is slowed, not cut off; a switch without meter support drops them instead. The rule expires on its own, after 10 s for a first offence and twice as long for each repeat, up to five minutes.

Table 12 compares a heavy flood with the meter off (five runs)

Table 12. A heavy flood (twelve workers at 0.1 s) with the flood meter off and on, BOTH-RAILS otherwise. Mean over runs with 95% t-interval; honest-device rows exclude every attacking device.

Metric	meter off (n=5)	meter on (n=4)
Honest mean latency	105.0 [101.2, 108.9] ms	**91.7** [89.4, 94.0] ms
Honest p95 latency	177.7 [160.9, 194.5] ms	**139.4** [132.9, 145.8] ms
Honest task success	98.91 [98.51, 99.31]%	98.79 [98.52, 99.06]%
Tasks served to flooder	8 951	**293**
Honest devices throttled	0	0

One meter-on run is excluded: the host suspended mid-run (recording span 24 331 s). Truncating it to its 316 s of normal recording gives the same reading (91.7→93.0 ms, 139.4→141.3 ms). [evidence/flood/summary_primary.txt]

and on (four usable runs). Unlike the anomaly gate, this response *does* improve service for honest devices: their mean latency falls by 13% and their p95 by 22%, with intervals that do not overlap, back to about the level seen without a heavy flood. Task success does not change. The flooder's served tasks fall by 97%, and no honest device was ever metered.

Two run-wide figures move the other way and must not be read as harms: total throughput falls from 41 to 15 tasks/s and run-wide mean latency rises, both because the flooder's own tasks dominate them. The flooder's connections that exceed the meter are dropped and retried by TCP, so its own tasks take seconds. This is the same two-population lesson as the fairness index above: report the attacker apart from the honest population.

VII-M. Detection quality

Scoring all 48 subjects — eight attackers and forty honest — against ground truth, over five runs per arm: overall accuracy is 91.67% in CONTROL, 97.92% in SCORE-ONLY and 97.50% [96.34, 98.66] in BOTH-RAILS. Every arm places $0/40$ honest subjects in an attack class, in every run. The acting arms name $8/8$ attacks in the right *family*; the control arm manages $4/8$, because two of its attackers are never contained and two never register on the rail that would name them.

Exact labels run 7.00 in SCORE-ONLY and 6.80 [6.24, 7.36] in BOTH-RAILS; the intervals overlap. The recurring miss is the blackhole labelled *grayhole* — the right family, one step too mild — because once it is quarantined at $\sim$10 s it receives almost no traffic and the timeout-rate tell never accumulates a measured rate.

The zero false-positive rate on forty honest subjects across fifteen runs is the number we would most want a reviewer to check, because it is what an over-eager detector buys its recall with. "No attack" is scored as a real class precisely so it cannot be hidden.

VII-N. Non-functional requirements

Over the five BOTH-RAILS runs: routing decision (packet-in → flow-mod) mean 0.54–0.55 ms, p95 0.71–0.73 ms, max 4.4–8.6 ms, against a < 200 ms requirement; isolation re-dispatch mean 12.9–23.5 ms, max 32.8 ms, against < 3000 ms; ledger overhead 0.045–0.052% of end-to-end task latency over 625–661 blocks, against $< 15\%$. [evidence/nfr_replicates.txt]

Two readings matter more than the verdicts. The re-dispatch is three orders of magnitude inside its bound, but the isolation time an operator experiences is dominated by *detection*, which is bounded below by the 1 s poll and the three-poll persistence requirement. Tightening it means polling faster or shortening persistence, and persistence is exactly what keeps healthy nodes out of quarantine on a jitter blip. And tamper-evidence at this batch size is effectively free.

The ledger can be replicated, and the report path does not wait for it. The controller can commit to a separate three-replica Raft cluster [30] instead of its in-process ledger. It is a client of the cluster: a background committer queues each batch off the report path, retries it until the cluster accepts it, and keeps a local copy of the chain that it re-validates block by block. A retried batch is recognised by a content hash and never becomes a second block. In a live run with this ledger we killed the leader with SIGKILL after 61 blocks: another replica was leader 0.47 s later, the one batch attempted during the election was retried, and all 649 submitted batches were committed exactly once (blocks 1–61 at Raft term 2, 62–649 at term 4), in order, with every block linked to the one before it. The killed replica, restarted a minute later from its on-disk log, caught up within 1.0 s. Commit time was 14 ms mean and 20 ms p95 (max 144 ms), and routing decisions stayed at 0.77 ms p95. [evidence/raft/failover_report.json, events_raft.jsonl] Replicas keep their term, vote and log on disk, so a restarted replica rejoins with its log. All ablation, sweep and flood runs in this paper used the in-process ledger; the replicated one is crash-fault tolerant, not Byzantine.

VII-O. Simulation: baselines and scale

Two questions need more repetitions or more nodes than a live testbed provides. Both drive the *real* selector, trust calculator and detectors inside a seeded simulation; neither is a live claim.

Against four routing baselines Five strategies × four scenarios × thirty seeds = 600 runs, paired on seed with common random numbers, tested with the Wilcoxon signed-rank test [36] and Holm–Bonferroni correction [37] within scenario. Fourteen of sixteen comparisons favour the full system, all at $p_{\text{adj}} = 7.45 \times 10^{-9}$ with rank-biserial $r = +1.00$. Under attack, SLO violations fall 15.1–29.8% against a *no-trust ablation* that is the same selector with only the trust term removed, and 61.8–68.8% against least-connections. [evidence/stats_slo.txt]

We attach a caveat to those p-values that we think should be standard. All thirty seeds won in every favouring comparison, so 7.45×10^{-9} is the *smallest value the test can return* at this n; it reflects a deterministic ordering, not a large sample. The effect sizes and medians carry the result, not the p-value.

The two losses are honest and instructive: in the *no-attack* scenario the system is 10.7% worse than round-robin and 4.8% worse than random. The cause is not the trust term but the staleness of the load signal every load-reactive strategy reads [23, 24]: a poll sweep shows the load-blind strategies flat and every load-reading strategy improving monotonically as the signal freshens, with the full system best on the board at a 0.01 s poll.

Selector scalability Sweeping $N = 4 \ldots 64$ at 60% offered load, power-of-two-choices scales linearly (48 → 762 task/s, delivery 99.7–99.8%) while arg max saturates at ∼85 task/s with delivery collapsing to 10.5%. Two qualifications keep this honest: arg max fails *differently* at the two ends of load — starving 13 of 16 nodes at 10% load, and at 60% ending up balanced *and* broken as queues push load off the favoured nodes — and p2c is **not** a per-decision CPU win, since both strategies cost 3–30 μs across the sweep because the selector ranks every eligible node either way to produce the decision's explanation. We claim the p2c win in fairness, delivery and delay, never in CPU. [evidence/scalability.csv]

The capacity cliff Isolating a node removes its capacity. At $N = 4$ with two attackers, isolating half the fleet leaves the honest half near saturation and the queueing wipes out the SLO gain — the system *loses* its headline metric while still delivering 32× fewer outright failures. By $N = 6$ the cliff is gone. **The defence needs enough spare capacity to absorb what it isolates**, which is a deployment constraint with a measured shape. [evidence/capacity_cliff/]

VIII. Discussion

VIII-A. Enforcement corrupts its own input signal

Data-plane enforcement has a consequence a simulated quarantine does not: installing a drop rule *tears down the in-flight TCP connections* to that node. The client waits out its own timeout on a dead socket while the controller's dispatch record has already moved to the survivor that inherited the flow. The resulting timeout is charged to a node that was never asked to do the work.

Uncorrected this closes a positive feedback loop — false attribution lowers an honest node's score, which quarantines it, which re-dispatches onto fewer nodes, which produces real timeouts, which feed the drop tell. The correction carries **no tuned threshold**: an inherited dispatch is stamped as such at re-assignment, and the report path skips trust attribution for it while *still publishing* the event, so availability and delivery ratio keep counting the client's real loss and the abstention is re-derivable offline. A proposal to instead excuse a re-steered outcome that arrived with "too little budget remaining" was tested against the recordings and would have excused zero of twenty-six cases: the median re-steered flow still had 2.86 of its 4.0 s remaining. It was a wrong-node problem, not a partial-budget problem.

Across the five BOTH-RAILS runs the rule fires on 123 of 32,107 reports, 0.38% (range 0.28–0.48%). [data/runs/treat/] That is a small number that decides whether honest nodes are quarantined, and no honest server was quarantined in any of the fifteen runs. The general lesson we would carry to any system enforcing a verdict in the data path: *the enforcement action is itself an input to the detector, and the coupling must be made explicit or it will be discovered as a cascade.* The same coupling surfaced twice more: under throttling, a quarantine stopped the completions the honesty check expected, so the check fired in reverse (§VII-F); and the gate's fast quarantine of a dropper starved the switch witness of the failures it needed until the witness judged recent connections only (§VII-G). And quarantine blocks the attestation channel: a quarantined server's traffic to the controller is dropped, attester included, so none of the run-time compromised server's 42 quotes arrived while it was quarantined, though it was quarantined for 49% of that time; each arrived within 2 s of its release (§VII-H). Harmless at 12 s quarantines against a 75 s admission, it would strand any server quarantined for longer, which could then never re-attest.

VIII-B. A retraction: containment does not cost attribution

An earlier version of this work claimed a *containment–attribution tension*: that isolating an attacker faster starves the evidence needed to name what it did, so defence quality and label precision trade against each other. The claim rested on three runs that differed in many things at once.

Replication does not support it. Over the ten acting runs, containment (mean attacker time isolated) spans 40.0% to 69.3% — a 29-point range — while exact labels are $7/8$ in nine runs and $6/8$ in one. The Spearman correlation is $\rho = -0.52$ with $p = 0.12$ at $n = 10$. A trade predicts $\rho < 0$, and the point estimate has that sign, but at this n nothing short of a clearly negative ρ with $p < 0.05$ would support it. More tellingly, the *direction* of the near-constant label accuracy across a large containment range is what one would expect if the effect were small or absent. **We withdraw the claim.** [evidence/replicates/summary.txt]

We report this at length for two reasons. It was one of two headline findings in the earlier draft, and a reader comparing versions deserves to know which way the evidence moved. And the failure mode is worth naming: three runs that differ in containment *and* in run length *and* in code version support almost any story one looks for.

What survives is narrower and still true: the specific mechanism is real — once the blackhole is quarantined its timeout-rate tell stops accumulating, and that is why it is labelled grayhole — but the effect does not show up as a measurable trade at this scale.

VIII-C. Route on what you verify

The split claim is not a quirk of our weights. It exists whenever a scheduler ranks servers on one self-reported figure and verifies a different one: the attacker lies in the figure that is used and tells the truth in the figure that is checked. In the switch-side attacks of Burke *et al.* [13] and Yu *et al.* [14] the misreported statistic is the only one the balancer sees, so detection must model its distribution [15]. A server's self-report is different: the controller holds an independent account of the same work, its own count of completed tasks, and the defence is to route on the quantity that account can confirm. We found the gap only because a liar was built to use it; a design review of the two code paths would not have flagged it, since each was correct on its own.

The ledger experiment (§VII-I) carries the same lesson up to the controller. Replicas can verify what a record lets them recompute, the arithmetic, and nothing else; evidence can only be checked against an independent account of the same events. Forged evidence also showed what is at stake: the controller's

own detectors read the evidence it forged, so the only check left standing was the one that did not depend on it.

VIII-D. What an ablation buys that a baseline does not

A comparison against an external baseline answers "is our system better than theirs?" An ablation answers "which of our parts is doing the work?", and only the second question has an actionable answer. Here the ablation and the sweep together produced three findings a baseline comparison would have missed: at the shipped threshold the gate is the only mechanism that contains half the server attacks; it contributes nothing measurable to the delivery and availability metrics a paper would naturally lead with; and the score, where it can be made to fire against those attackers at all, does so only inside a narrow band and several times more slowly.

The ablation was cheap only because of the architectural rule in §IV: the request path writes evidence and the control loop reads decisions through separate code paths, so enforcement can be removed without touching the trust engine. We would recommend that separation to anyone building a system of this shape, for the evaluation it makes possible as much as for the design.

One implementation detail carried the validity of the whole comparison, and it is the kind of thing that is easy to get silently wrong. Disabling the gate by raising its threshold to infinity would also have removed $\bar{A}$ from Eq. 1, changing the score itself and making the arms measure different quantities. Separating *computing* $\bar{A}$ from *acting* on it is what keeps the ablation an ablation.

IX. Threats to Validity

Internal Five runs per arm is enough for between-run intervals and not enough for distributional claims; we report intervals and decline significance tests between arms on service metrics. Runs were executed in rotating arm order on one otherwise-idle host, which controls for monotone drift but not for a change that coincides with one arm's slots. The three arms share a codebase, so a defect in the shared path would move all three together — the ablation is protected against confounds *between* arms, not against a common-mode error.

The probation path Two such common-mode defects were found by the hold experiment (§VII-J) and affect every earlier recording that contains probation trials: the score-only, sweep, both-rails and flood arms, 35 runs. First, quarantine installs its drop rules on every switch, but a trial's rules out-ranked them only on the client's ingress switch, so a trial reached the quarantined server only from a client behind that server's own switch: 304 of the 3 816 trials in those runs, 8.0%. Second, a trial's rules expired after exactly the client's 4 s timeout, so when a trial timed out the client's closing packet reached the controller as a new connection, was routed to another server, and the trial's report was charged to that server: 43 times in the 35 runs, 34 of them to an honest server. Neither touches when an attacker was first quarantined or which rail fired, since no trial precedes a first quarantine and a server the gate has flagged is never probed. Trust-quarantined attackers were released more slowly than designed, so the time a trust-rail grayhole spent isolated is an upper bound. No honest server was quarantined by either defect. But the three honest dips of §VII-D, to 0.417–0.421, are each one of those misattributed timeouts, within a second of it; without them no honest server fell below 0.50 after onset in the 25 score-only, sweep and both-rails runs. The dips still measure what one charged timeout does to an honest score, and a real timeout does the same, but their cause is the defect, not load. Both defects are fixed: a trial now opens a passage on every switch, and only a SYN opens a connection. In the final hold batch every trial was judged and none was charged elsewhere. The earlier arms were not re-run.

Construct The central measurement, whether the score ever crossed θ, is threshold-relative. We therefore swept θ live (§VII-D); the sweep has five runs per threshold, so the narrowness of the band rests on three honest-server dips observed in other arms, each one misattributed timeout (see *The probation path* above), and on what one charged timeout does to an honest score, not on a measured rate of false quarantine at each threshold. Isolation stands in for the wireless-sensor notion of "node death" in our availability metric because no energy model exists here and inventing one for mains-powered servers would be a fabricated number; readers comparing against WSN lifetime literature should treat the metric as redefined, not transferred.

External An emulated network [4] on a four-core host, not a field deployment. Scalability beyond eight servers is simulation driving the real selector and supports a claim about the *selection rule*, not a capacity figure for real machines. The Raft measurements are three OS processes over loopback TCP with real process kills: the code, sockets and process boundaries are real; only hardware separation is not. One workload shape, Poisson-like at a fraction of farm capacity; bursty and heavy-tailed workloads are untested.

Conclusion The simulation p-values are the smallest the test can return at $n = 30$ and should be read as a deterministic ordering, not a large sample (§VII-O). The live results carry intervals rather than p-values by design. Where arms overlap we say so rather than describing the point estimates.

Adversary The attackers were built by the same team as the detectors, and the load-misreporting server of the ablation slows itself by burning CPU, which is exactly what the gate's latency tell detects (§V-B). Liars that serve at honest speed were tested separately (§VII-E), at one raised occupancy reached by holding workers rather than by CPU work, so servers there are occupied but not computing; a CPU-bound workload at that occupancy does not fit the four-core host. Their attacker model is **adaptive only within one rule**: the adaptive liar tracks its peers' claims and the honesty threshold, whereas security evaluations are expected to include adaptive attackers [38, 39] that search the defence. An adversary that observes its own detection and

modulates — keeping the claimed-versus-observed gap just under the honesty normaliser, or timing an on-off period against the EMA window — would evade the tells we implement. Our claim is bounded to the seven behaviours in §III. Notably, §V-B shows the honesty term already barely registers a *non*-adaptive liar, so the margin an adaptive one would need is small.

X. Limitations and Future Work

1. **Liars that serve at honest speed were tested at two occupancies** (0.01 and 0.40; §VII-E) with eight servers. Under the verified-claim default a liar's only lever is its busy-time counter, which the honesty check bounds; how much traffic an attacker that tunes that counter against the threshold can win, across occupancies, is not measured.
2. **Five runs per arm**, one host, one topology size, one workload shape. The false-quarantine rate at each threshold is not measured directly.
3. **No adaptive adversary.** A server that watches its own anomaly level and stays just under the gate is the most valuable next attacker.
4. **The controller is a trusted single point**, in tension with the doctrine the system implements, and the replicated ledger tolerates crashes, not malicious replicas. Replicas that re-verify the trust arithmetic catch a forged score after the fact, but not forged evidence, and they do not stop the controller routing on what it forged.
5. **Hand-set weights.** The online bandit explores a fixed set of weight presets; whether learned weights beat hand-tuned ones is unanswered.
6. **PRESENT-80 with a fleet-wide key is an architecture demonstration**, not a security claim; per-device keys are implemented but were not used in these runs.
7. **The drop tell judges on as few as four outcomes.** In a synthetic four-server replay with an injected 8% timeout rate it quarantined an honest server in 5 of 20 seeds under verified ranking and 1 of 20 under raw ranking (not significant, $p \approx 0.18$); no live run showed it.
8. **The switch witness judges one window.** Acting on its verdicts, it took one false action in 73 runs: a burst of failures caused by a host-wide stall, which a rule requiring persistence across windows would have discounted. Holding a blamed server costs trials, each a lost task, against droppers the gate already contains, and releases a 50% grayhole after about fourteen. All clients of one switch lying together is indistinguishable from the switch lying, and a switch covering for a dropper that the gate isolates quickly goes unblamed.
9. **The unreachable signal has no persistence**: a single `/status` poll slower than 0.5 s counts, which is how a server throttled to 10% was quarantined for 3.6 s. Mixed hardware was emulated with CPU caps on one host; physical hardware is untested.
10. **Attestation is software.** The quote is an Ed25519 signature made by a root process on the same host as the agent, not a hardware root of trust; the measurement covers the interpreter and the agent's two source files, not its libraries, configuration or command line, and the tampered image differs by one appended line. It shows the mechanism and how it complements behaviour, not resistance to a determined attacker. Attestation should also travel on a management path quarantine does not cut.

The open question this work leaves is how much evidence a gate needs before an adaptive adversary can starve it, and how small a score's margin can become before a criteria-based rule is the only workable one.

XI. Reproducibility

Every number carries the artifact it was read from. The offline scorers read the same append-only recordings the live dashboard streams, and none needs the controller stack installed. 328 run recordings are archived under `data/runs/<arm>/`; one aggregator produces every live figure in §VII. The code that produced the ablation and sweep runs is commit `8998a7c`; the flood runs, `7a2d795`; the four liar batches, `3b95031`, `4dfbdc2`, `ff64b87` and `fa37a3a`; the honest-slow runs, `d1076c4`; the two witness batches, `0c8d064` and `15728ac`; the attestation runs, `840c58f`; the ledger runs, `257b5ed`; the acting witness, `13006e8`; the three hold batches, `1dd5a8d`, `284ecf4` (8 of 25 runs completed) and `26e2b04`. Each run's commit is recorded in the runner's manifest. Each batch's prediction is a file committed before the batch ran (`docs/predictions/`). At the time of writing the test suite reports 1 461 passing and 11 skipped, with no failures. Commands, with `EV` for `evaluation`:

Result	Command (prefix `python3 -m`)
Tables 2–11	`EV.replicates data/runs -arms base,scoreonly,treat`
Table 5	`EV.replicates data/runs -arms scoreonly,scoreonly_t040, scoreonly_t045,scoreonly_t050, treat`
Table 12	`EV.replicates data/runs -arms flood_off,flood_on`
Table 6	`EV.replicates data/runs -arms liar_c_treat,liarfix_t_treat,...` (four batches; `tools/regenerate.sh`)
Liar model	`EV.liar_sweep -seeds 5`
§VII-F	`EV.replicates data/runs -arms hw_fleet,hw_class,...`
Table 7	`EV.witness_report data/runs -arms wit_control,...,witfix_collude`
Witness, offline	`EV.witness_offline data/runs -arms ...` (110 recordings; `tools/regenerate.sh`)
Table 8	`EV.attestation_report data/runs -arms att_off_clean,..., att_on_runtime`
Table 9	`EV.ledger_report data/runs -arms lv_noverify,...,lv_forge_evidence`
Table 10	`EV.witness_act_report data/runs -arms wa_forge_observe, wh3_forge_act,...,wh3_drop_hold`
§VII-N	`EV.nfr_report data/runs/treat/rNN.jsonl`
§VII-O baselines	`EV.baseline -runs 30 -csv r.csv` `EV.stats r.csv`
§VII-O scale	`EV.scalability_sweep -ns 4,8,16,32,64`
The live runs themselves	`sudo ARMS="..." tools/run_replicates.sh` (forgery arms also need `ZTSDN_ALLOW_FORGERY=1`)

Live runs require root, Open vSwitch and Mininet, and overwrite the active recording at start-up; the runner archives each run before the next begins. *A recording that is not archived the moment a run finishes cannot be re-scored*, a discipline this project learned by losing one.

XII. Conclusion

Zero Trust requires continuous verification and grants no standing trust. Applied to the servers of an edge network, it raises a concrete question: can a weighted trust score do the verifying on its own? Against attackers that fail tasks, yes: the score isolated them as fast as an independent gate. Against attackers that complete every task while misreporting their load, only barely. At the shipped threshold the score never isolated them, in any of fifteen runs. Raising the threshold made it fire, but only in a band a few hundredths wide above their floor of about 0.40, close to where one charged timeout puts an honest server and just below the 0.5 every server starts at, and three to twenty times more slowly than the gate, which isolated them in every run at trust scores near 0.71.

The attacker that tested this was slow, and its lie did not attract traffic. Liars that serve at honest speed changed the picture. A lie that is internally consistent does not pay against this design: the honesty term and power-of-two-choices together cost it more than its claim wins. A lie split between the figure the router ranks on and the figure the honesty check verifies does pay, by a third more traffic than an honest server, and neither rail sees it; ranking on the verified figure closes it. Building that liar also exposed a defect in the honesty check that only load reveals.

Four further checks bound the claims. Honest slowness, emulated as mixed hardware and thermal throttling, is not mistaken for lying at this load; the one false quarantine came through a signal without persistence, not through the latency tell. The switches' own counters give a third account of every task that, joined with the clients' reports, names whether a server, a switch or a client is lying, without blaming an honest party in 25 runs. And attestation and behaviour cover each other's blind spot: a tampered server is refused before it serves anything, while a compromise of genuine code at run time passes attestation and is caught only by behaviour. Finally, ledger replicas that replay the trust arithmetic reject every forged score that reaches them, but not forged evidence, which blinded the controller's own detectors and was exposed only by the switches. Acting on the switches' verdicts, and holding a blamed server until they see it serve, brought honest task success under that forgery from 82% back to 99%.

We report equally what enforcement did not buy. The gate improved containment and the latency tail but not delivery, throughput or honest availability, and a containment-versus-attribution trade we previously claimed did not survive replication. A second data-plane response, a per-client meter on a flooding device, did improve honest service: under a heavy flood it brought honest latency back to its unflooded level while cutting the flooder's service by 97%.

Two design decisions made these measurements cheap: enforcement could be removed without touching the trust engine, and disabling the gate's *action* was kept separate from removing its *term* from the score. A system of this shape should make both early, for the evaluation they make possible as much as for the architecture.

Data Availability

The 328 live recordings, the aggregator output, the experiments' registered predictions, and the simulation CSVs behind every table are archived with the project repository under `data/runs/`, `data/replicates/` and `paper-draft/evidence/`. Recordings are JSONL event streams of ∼130 MB each. TODO BEFORE SUBMISSION: deposit at a persistent archive and cite the DOI here.

Ethics

No human subjects, no personal data, and no third-party network were involved. All attacks were executed against an emulated network on a single host under the authors' control. No institutional review was required.

Author Contributions (CRediT)

TODO BEFORE SUBMISSION: CONFIRM PER-AUTHOR ROLES. The intended statement is: *Conceptualization, Methodology* — all authors; *Software, Investigation, Data curation* — all authors;

Formal analysis, Visualization, Writing — original draft — all authors; *Supervision* — R. N.

Declaration of Generative AI Use

An AI coding assistant (Claude, Anthropic) was used in this work: for implementing parts of the evaluation tooling. All experiments were designed by the authors and executed on the authors' hardware, and every reported number was produced by the cited tool from an archived recording rather than generated by a model. The authors verified the reference list and are responsible for all content.

Conflicts of Interest

The authors declare no competing interests.

Funding

This work received no external funding.

Acknowledgements

We thank our faculty guide for continued supervision. The substitution of the maintained Ryu fork for the framework originally specified was an environment-driven engineering decision documented in the project repository.

References

[1] S. Rose, O. Borchert, S. Mitchell, and S. Connelly. Zero trust architecture. Technical Report NIST Special Publication 800-207, National Institute of Standards and Technology, August 2020.

[2] M. Huang, Z. Li, F. Xiao, S. Long, and A. Liu. Trust mechanism-based multi-tier computing system for service-oriented edge-cloud networks. *IEEE Transactions on Dependable and Secure Computing*, 21(4):1639–1651, 2024.

[3] B. Ali, M. A. Gregory, and S. Li. Trust-aware task load balancing in multi-access edge computing based on blockchain and a zero trust security capability framework. *Transactions on Emerging Telecommunications Technologies*, 34(12):e4845, 2023.

[4] B. Lantz, B. Heller, and N. McKeown. A network in a laptop: Rapid prototyping for software-defined networks. In *ACM SIGCOMM HotNets*, 2010.

[5] A. Iftikhar, F. B. Hussain, K. N. Qureshi, M. Shiraz, and M. Sookhak. Securing edge based smart city networks with software defined networking and zero trust architecture. *Journal of Network and Computer Applications*, 2025.

[6] ETSI. Multi-access edge computing (mec); framework and reference architecture. Technical Report GS MEC 003 V3.1.1, ETSI, 2022.

[7] R. Roman, J. Lopez, and M. Mambo. Mobile edge computing, fog et al.: A survey and analysis of security threats and challenges. *Future Generation Computer Systems*, 78:680–698, 2018.

[8] Y. Xiao, Y. Jia, C. Liu, X. Cheng, J. Yu, and W. Lv. Edge computing security: State of the art and challenges. *Proceedings of the IEEE*, 107:1608, 2019.

[9] D. Kreutz, F. M. V. Ramos, and P. Veríssimo. Towards secure and dependable software-defined networks. In *ACM SIGCOMM Workshop on Hot Topics in Software Defined Networking (HotSDN)*, 2013.

[10] S. Scott-Hayward, S. Natarajan, and S. Sezer. A survey of security in software defined networks. *IEEE Communications Surveys & Tutorials*, 18(1):623–654, 2016.

[11] H. Birkholz et al. Remote ATtestation procedures (RATS) architecture. Technical Report RFC 9334, Internet Engineering Task Force, 2023.

[12] C. Buck, C. Olenberger, A. Schweizer, F. Völter, and T. Eymann. Never trust, always verify: A multivocal literature review on current knowledge and research gaps of zero-trust. *Computers & Security*, 110:102436, 2021.

[13] Q. Burke, P. McDaniel, T. La Porta, M. Yu, and T. He. Misreporting attacks against load balancers in software-defined networking. *Mobile Networks and Applications*, 28(4):1482–1497, 2023.

[14] M. Yu, Q. K. Burke, T. La Porta, and P. McDaniel. Stealthy misreporting attacks against load balancing. *IEEE/ACM Transactions on Networking*, 32(4):3622–3635, 2024.

[15] S. Saha, M. N. Absur, S. Yousefi, and S. Debroy. Detection of misreporting attacks on software-defined immersive environments. arXiv:2509.18040, 2025.

[16] M. Dhawan, R. Poddar, K. Mahajan, and V. Mann. SPHINX: Detecting security attacks in software-defined networks. In *Network and Distributed System Security Symposium (NDSS)*, 2015.

[17] P. Huang, C. Guo, L. Zhou, J. R. Lorch, Y. Dang, M. Chintalapati, and R. Yao. Gray failure: The Achilles' heel of cloud-scale systems. In *Workshop on Hot Topics in Operating Systems (HotOS)*, 2017.

[18] P. Huang et al. Capturing and enhancing in situ system observability for failure detection. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018.

[19] R. Lu et al. Perseus: A fail-slow detection framework for cloud storage systems. In *USENIX Conference on File and Storage Technologies (FAST)*, 2023.

[20] Envoy Project. Outlier detection – envoy proxy documentation. `https://envoyproxy.io/docs/envoy/latest/intro/arch_overview/upstream/outlier`, 2026. Accessed 14 October 2026.

[21] M. Mitzenmacher. The power of two choices in randomized load balancing. *IEEE Transactions on Parallel and Distributed Systems*, 12(10):1094–1104, 2001.

[22] Y. Azar, A. Z. Broder, A. R. Karlin, and E. Upfal. Balanced allocations. *SIAM Journal on Computing*, 29(1):180–200, 1999.

[23] M. Mitzenmacher. How useful is old information? *IEEE Transactions on Parallel and Distributed Systems*, 11(1):6–20, 2000.

[24] M. Dahlin. Interpreting stale load information. *IEEE Transactions on Parallel and Distributed Systems*, 11(10):1033–1047, 2000.

[25] R. Jain, D.-M. Chiu, and W. Hawe. A quantitative measure of fairness and discrimination for resource allocation in shared computer systems. Technical Report DEC Research Report TR-301, Digital Equipment Corporation, 1984.

[26] J. D. C. Little. A proof for the queuing formula: $l = \lambda w$. *Operations Research*, 9(3):383–387, 1961.

[27] A. Bogdanov, L. R. Knudsen, G. Leander, C. Paar, A. Poschmann, M. J. B. Robshaw, Y. Seurin, and C. Vikkelsoe. PRESENT: An ultra-lightweight block cipher. In *Cryptographic Hardware and Embedded Systems (CHES 2007)*, volume 4727 of *LNCS*, pages 450–466, 2007.

[28] ISO/IEC. Information security — lightweight cryptography — part 2: Block ciphers. Technical Report ISO/IEC 29192-2:2019, International Organization for Standardization, 2019.

[29] R. C. Merkle. A digital signature based on a conventional encryption function. In *Advances in Cryptology — CRYPTO '87*, volume 293 of *LNCS*, pages 369–378, 1988.

[30] D. Ongaro and J. Ousterhout. In search of an understandable consensus algorithm. In *USENIX Annual Technical Conference (ATC)*, pages 305–319, 2014.

[31] P. Auer, N. Cesa-Bianchi, and P. Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256, 2002.

[32] M. T. Nygard. *Release It!: Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, 2nd edition, 2018.

[33] J. R. Douceur. The Sybil attack. In *Peer-to-Peer Systems (IPTPS 2002)*, volume 2429 of *LNCS*, pages 251–260, 2002.

[34] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner. OpenFlow: Enabling innovation in campus networks. *ACM SIGCOMM Computer Communication Review*, 38(2):69–74, 2008.

[35] E. B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

[36] F. Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.

[37] S. Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[38] N. Carlini et al. On evaluating adversarial robustness. arXiv:1902.06705, 2019.

[39] F. Tramèr, N. Carlini, W. Brendel, and A. Mądry. On adaptive attacks to adversarial example defenses. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv:2002.08347.